

Математическое обоснование работы паттрейсера на видеокарте

1 Введение и архитектура

1.1 Обзор GPU паттрейсинга

Path tracing (паттрейсинг) — это метод фотореалистичного рендеринга, основанный на решении уравнения рендеринга методом Монте-Карло. Выполнение паттрейсинга на GPU (Graphics Processing Unit) позволяет значительно ускорить вычисления благодаря массивному параллелизму.

Основная идея заключается в том, что каждый пиксель изображения обрабатывается независимо, что идеально подходит для параллельной архитектуры GPU. Каждый поток GPU вычисляет цвет одного пикселя, трассируя множество случайных лучей и усредняя результаты.

1.2 Архитектура OpenGL Compute Shaders

В данной реализации используется OpenGL 4.3+ с поддержкой compute shaders. Compute shader — это специальный тип шейдера, который позволяет выполнять произвольные вычисления на GPU без привязки к графическому конвейеру.

Архитектура compute shader основана на концепции work groups (рабочих групп). Каждая рабочая группа содержит фиксированное количество потоков (threads), которые выполняются параллельно. В нашем случае используется размер рабочей группы $16 \times 16 = 256$ потоков.

1.3 Структура данных на GPU

Данные передаются на GPU через специальные буферы:

- **Uniform Buffer Objects (UBO)** — для постоянных данных (камера, небо, туман)
- **Shader Storage Buffer Objects (SSBO)** — для массивов данных (материалы, объекты, накопительный буфер)

- **Image2D** — для выходного изображения (текстура RGBA16F)

2 Форматы данных и передача на GPU

2.1 Uniform Buffer Objects (UBO) — формат std140

Uniform Buffer Objects используют выравнивание std140, которое гарантирует определенную структуру размещения данных в памяти. Это необходимо для корректного доступа к данным из шейдера.

2.1.1 Блок камеры

Камера передается как структура из 16 float32 (4 vec4), что составляет 64 байта:

$$\text{CameraBlock} = \begin{bmatrix} \text{camPos} & (\text{vec4}, 16 \text{ байт}) \\ \text{camTarget} & (\text{vec4}, 16 \text{ байт}) \\ \text{camUp} & (\text{vec4}, 16 \text{ байт}) \\ \text{camFov, camAperture, camFocusDist, camAspect} & (4 \text{ float}, 16 \text{ байт}) \end{bmatrix} \quad (1)$$

Математически позиция камеры $\mathbf{c}_{\text{pos}}$, цель $\mathbf{c}_{\text{target}}$ и вектор вверх $\mathbf{c}_{\text{up}}$ определяют систему координат камеры.

2.1.2 Блок неба

Небо передается как структура с выравниванием std140:

$$\text{SkyBlock} = \begin{bmatrix} \text{skyType} & (\text{float}, 4 \text{ байта} + 12 \text{ байт padding}) \\ \text{skyColor} & (\text{vec4}, 16 \text{ байт}) \\ \text{skyHorizon} & (\text{vec4}, 16 \text{ байт}) \\ \text{skyZenith} & (\text{vec4}, 16 \text{ байт}) \end{bmatrix} \quad (2)$$

Общий размер: 64 байта (выравнивание std140 требует, чтобы каждый элемент занимал кратные 16 байт).

2.1.3 Блок тумана

Туман содержит физические параметры объемного рассеяния:

$$\text{FogBlock} = \begin{bmatrix} \text{fogDensity, fogScatter, fogAffectSky, fogGpuVolumetric} & (4 \text{ float}) \\ \text{fogColor} & (\text{vec4}) \\ \text{fogSigmaS, fogSigmaA, fogG} & (3 \text{ float} + \text{padding}) \\ \text{fogHeteroStrength, fogNoiseScale, fogNoiseOctaves} & (3 \text{ float} + \text{padding}) \end{bmatrix} \quad (3)$$

Ключевые физические параметры:

- σ_s — коэффициент рассеяния (scattering coefficient)
- σ_a — коэффициент поглощения (absorption coefficient)
- $\sigma_t = \sigma_s + \sigma_a$ — коэффициент экстинкции (extinction coefficient)
- g — параметр анизотропии фазовой функции Хеньи-Грина

2.2 Shader Storage Buffer Objects (SSBO) — формат std430

SSBO используют более компактное выравнивание std430, которое не требует padding между элементами массивов.

2.2.1 Буфер материалов

Каждый материал занимает 20 float32 (80 байт):

$$\text{matData}[i] = \begin{bmatrix} \text{type} & (\text{float, индекс } 0) \\ \text{rough} & (\text{float, индекс } 1) \\ \text{ior} & (\text{float, индекс } 2) \\ \text{smoothness} & (\text{float, индекс } 3) \\ \text{albedo.r, albedo.g, albedo.b} & (\text{float, индексы } 4-6) \\ \text{reflectivity} & (\text{float, индекс } 7) \\ \text{emit.r, emit.g, emit.b} & (\text{float, индексы } 8-10) \\ \text{pad0} & (\text{float, индекс } 11) \\ \text{absorption.r, absorption.g, absorption.b} & (\text{float, индексы } 12-14) \\ \text{absorptionScale} & (\text{float, индекс } 15) \\ \text{tint.r, tint.g, tint.b} & (\text{float, индексы } 16-18) \\ \text{pad2} & (\text{float, индекс } 19) \end{bmatrix} \quad (4)$$

Размер буфера материалов:

$$\text{size}_{\text{materials}} = N_{\text{materials}} \times 20 \times 4 \text{ байт} = 80N_{\text{materials}} \text{ байт}$$

где $N_{\text{materials}}$ — количество материалов в сцене.

2.2.2 Буфер объектов

Каждый объект занимает 12 float32 (48 байт):

$$\text{objData}[i] = \begin{bmatrix} \text{type} & (\text{float}, \text{индекс } 0) \\ \text{materialIndex} & (\text{float}, \text{индекс } 1) \\ \text{pad0, pad1} & (\text{float}, \text{индексы } 2-3) \\ \text{pos.x, pos.y, pos.z} & (\text{float}, \text{индексы } 4-6) \\ \text{pad2} & (\text{float}, \text{индекс } 7) \\ \text{size.x, size.y, size.z} & (\text{float}, \text{индексы } 8-10) \\ \text{pad3} & (\text{float}, \text{индекс } 11) \end{bmatrix} \quad (5)$$

Размер буфера объектов:

$$\text{size}_{\text{objects}} = N_{\text{objects}} \times 12 \times 4 \text{ байт} = 48N_{\text{objects}} \text{ байт}$$

2.2.3 Буфер индексов источников света

Список индексов эмиссивных объектов хранится как массив int32:

$$\text{lightIndices} = [i_1, i_2, \dots, i_{N_{\text{lights}}}]$$

где i_j — индекс объекта-источника света в массиве объектов.

Размер:

$$\text{size}_{\text{lights}} = N_{\text{lights}} \times 4 \text{ байт}$$

2.2.4 Накопительный буфер

Накопительный буфер хранит сумму всех сэмплов для каждого пикселя:

$$\text{accumData}[i] = [R_i, G_i, B_i]$$

где $i = y \times W + x$ — линейный индекс пикселя с координатами (x, y) .

Размер накопительного буфера:

$$\text{size}_{\text{accum}} = W \times H \times 3 \times 4 \text{ байт} = 12WH \text{ байт}$$

где W — ширина изображения, H — высота изображения.

3 Распараллеливание вычислений

3.1 Work Group размер

Compute shader использует фиксированный размер рабочей группы (work group):

$$\text{local_size_x} = 16, \quad \text{local_size_y} = 16$$

Это означает, что каждая рабочая группа содержит $16 \times 16 = 256$ потоков, которые выполняются параллельно на одном вычислительном блоке GPU.

3.2 Диспетчеризация compute shader

Количество рабочих групп вычисляется по формулам:

$$\text{groupsX} = \left\lceil \frac{W}{16} \right\rceil = \left\lfloor \frac{W + 15}{16} \right\rfloor \quad (6)$$

$$\text{groupsY} = \left\lceil \frac{H}{16} \right\rceil = \left\lfloor \frac{H + 15}{16} \right\rfloor \quad (7)$$

где $\lceil \cdot \rceil$ — функция округления вверх (ceiling), $\lfloor \cdot \rfloor$ — округление вниз (floor).

Общее количество рабочих групп:

$$N_{\text{groups}} = \text{groupsX} \times \text{groupsY}$$

Общее количество потоков (может превышать количество пикселей):

$$N_{\text{threads}} = N_{\text{groups}} \times 256$$

3.3 Глобальный индекс пикселя

Каждый поток имеет уникальный глобальный индекс, вычисляемый как:

$$\text{pix}_x = \text{gl_GlobalInvocationID.x} \quad (8)$$

$$\text{pix}_y = \text{gl_GlobalInvocationID.y} \quad (9)$$

где `gl_GlobalInvocationID` — встроенная переменная OpenGL, содержащая глобальные координаты потока.

Линейный индекс пикселя:

$$\text{pixIdx} = \text{pix}_y \times W + \text{pix}_x$$

3.4 Инициализация генератора случайных чисел

Для обеспечения детерминированности и воспроизводимости результатов, каждый поток инициализирует свой генератор случайных чисел на основе координат пикселя и seed кадра:

$$\text{state}_0 = \text{hash}(\text{pix}_x \times 1973 \oplus \text{pix}_y \times 9277 \oplus \text{frameSeed})$$

где $\oplus$ — операция XOR, hash — хэш-функция для генерации псевдослучайных чисел.

Хэш-функция реализуется как последовательность битовых операций:

$$x_1 = x \oplus (x \gg 17) \quad (10)$$

$$x_2 = x_1 \times 0xED5AD4BB \quad (11)$$

$$x_3 = x_2 \oplus (x_2 \gg 11) \quad (12)$$

$$x_4 = x_3 \times 0xAC4C1B51 \quad (13)$$

$$x_5 = x_4 \oplus (x_4 \gg 15) \quad (14)$$

$$\text{hash}(x) = (x_5 \times 0x31848BAB) \oplus (x_5 \gg 14) \quad (15)$$

где $\gg$ — побитовый сдвиг вправо.

4 Генерация первичных лучей

4.1 Построение системы координат камеры

Система координат камеры определяется тремя ортогональными векторами:

$$\mathbf{w} = \frac{\mathbf{c}_{\text{pos}} - \mathbf{c}_{\text{target}}}{\|\mathbf{c}_{\text{pos}} - \mathbf{c}_{\text{target}}\|} \quad (16)$$

$$\mathbf{u} = \frac{\mathbf{c}_{\text{up}} \times \mathbf{w}}{\|\mathbf{c}_{\text{up}} \times \mathbf{w}\|} \quad (17)$$

$$\mathbf{v} = \mathbf{w} \times \mathbf{u} \quad (18)$$

где $\mathbf{w}$ направлен от цели к позиции камеры (обратное направление взгляда), $\mathbf{u}$ — вектор вправо, $\mathbf{v}$ — вектор вверх.

4.2 Вычисление размеров viewport

Размеры viewport вычисляются на основе поля зрения (FOV) и соотношения сторон:

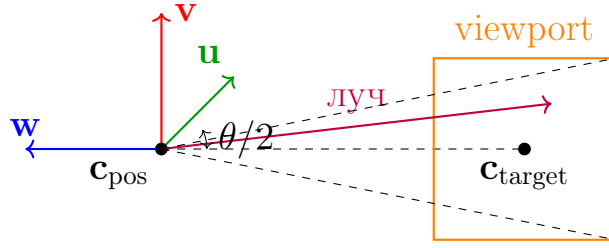


Рис. 1: Система координат камеры и генерация луча

$$\theta = \frac{\text{FOV} \times \pi}{180} \quad (\text{преобразование из градусов в радианы}) \quad (19)$$

$$h = \tan\left(\frac{\theta}{2}\right) \quad (20)$$

$$\text{viewportHeight} = 2h \quad (21)$$

$$\text{viewportWidth} = \text{aspect} \times \text{viewportHeight} \quad (22)$$

где $\text{aspect} = W/H$ — соотношение сторон изображения.

4.3 Построение луча без глубины резкости

Для простой камеры (без DOF) луч строится следующим образом:

$$u = \frac{\text{pix}_x + \xi_x}{W - 1} \quad (\text{нормализованные координаты } [0, 1]) \quad (23)$$

$$v = \frac{H - 1 - \text{pix}_y + \xi_y}{H - 1} \quad (24)$$

где $\xi_x, \xi_y \in [0, 1]$ — случайные смещения для сэмплирования внутри пикселя.

Направление луча:

$$\mathbf{horizontal} = \text{viewportWidth} \times \text{focusDist} \times \mathbf{u} \quad (25)$$

$$\mathbf{vertical} = \text{viewportHeight} \times \text{focusDist} \times \mathbf{v} \quad (26)$$

$$\mathbf{lowerLeftCorner} = \mathbf{c}_{\text{pos}} - \frac{1}{2}\mathbf{horizontal} - \frac{1}{2}\mathbf{vertical} - \text{focusDist} \times \mathbf{w} \quad (27)$$

$$\mathbf{dir} = \mathbf{lowerLeftCorner} + u \times \mathbf{horizontal} + v \times \mathbf{vertical} - \mathbf{c}_{\text{pos}} \quad (28)$$

$$\mathbf{r}_{\text{dir}} = \frac{\mathbf{dir}}{\|\mathbf{dir}\|} \quad (29)$$

Начало луча:

$$\mathbf{r}_{\text{orig}} = \mathbf{c}_{\text{pos}}$$

4.4 Генерация луча с учетом глубины резкости (DOF)

Для камеры с диафрагмой ($\text{aperture} > 0$) луч генерируется с учетом глубины резкости:

$$\text{lensRadius} = \frac{\text{aperture}}{2} \quad (30)$$

$$\mathbf{rd} = \text{lensRadius} \times \text{randomInUnitSphere}() \quad (31)$$

$$\mathbf{offset} = \mathbf{u} \times \mathbf{rd}_x + \mathbf{v} \times \mathbf{rd}_y \quad (32)$$

$$\mathbf{r}_{\text{orig}} = \mathbf{c}_{\text{pos}} + \mathbf{offset} \quad (33)$$

$$\mathbf{dir} = \mathbf{lowerLeftCorner} + u \times \mathbf{horizontal} + v \times \mathbf{vertical} - \mathbf{r}_{\text{orig}} \quad (34)$$

$$\mathbf{r}_{\text{dir}} = \frac{\mathbf{dir}}{\|\mathbf{dir}\|} \quad (35)$$

4.5 Stratified Sampling

Для уменьшения дисперсии (variance) используется стратифицированное сэмплирование. Пиксель разбивается на $N \times N$ страт (обычно $N = 4$, что дает 16 страт):

$$\text{strataSize} = 4 \quad (36)$$

$$\text{totalStrata} = \text{strataSize}^2 = 16 \quad (37)$$

$$\text{samplesPerStratum} = \left\lfloor \frac{N_{\text{samples}}}{\text{totalStrata}} \right\rfloor \quad (38)$$

$$\text{extraSamples} = N_{\text{samples}} - \text{samplesPerStratum} \times \text{totalStrata} \quad (39)$$

Для каждой страты (s_x, s_y) генерируются случайные смещения:

$$\xi_x, \xi_y \sim \mathcal{U}(0, 1) \quad (\text{равномерное распределение}) \quad (40)$$

$$\text{stratumU} = \frac{s_x + \xi_x}{\text{strataSize}} \quad (41)$$

$$\text{stratumV} = \frac{s_y + \xi_y}{\text{strataSize}} \quad (42)$$

$$u = \frac{\text{pix}_x + \text{stratumU}}{W - 1} \quad (43)$$

$$v = \frac{H - 1 - \text{pix}_y + \text{stratumV}}{H - 1} \quad (44)$$

5 Пересечение лучей с объектами

5.1 Пересечение со сферой

Луч задается параметрическим уравнением:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \quad t > 0$$

где $\mathbf{o}$ — начало луча, $\mathbf{d}$ — направление луча (нормализовано).

Сфера задается центром $\mathbf{c}$ и радиусом r . Подставляя уравнение луча в уравнение сферы:

$$\|\mathbf{r}(t) - \mathbf{c}\|^2 = r^2$$

Раскрывая скобки:

$$\|\mathbf{o} + t\mathbf{d} - \mathbf{c}\|^2 = r^2 \quad (45)$$

$$\|\mathbf{o} - \mathbf{c} + t\mathbf{d}\|^2 = r^2 \quad (46)$$

$$(\mathbf{o} - \mathbf{c} + t\mathbf{d}) \cdot (\mathbf{o} - \mathbf{c} + t\mathbf{d}) = r^2 \quad (47)$$

Раскрывая скалярное произведение:

$$\|\mathbf{o} - \mathbf{c}\|^2 + 2t(\mathbf{o} - \mathbf{c}) \cdot \mathbf{d} + t^2\|\mathbf{d}\|^2 = r^2 \quad (48)$$

Поскольку $\mathbf{d}$ нормализован, $\|\mathbf{d}\|^2 = 1$. Получаем квадратное уравнение:

$$at^2 + bt + c = 0 \quad (49)$$

где:

$$a = \|\mathbf{d}\|^2 = 1 \quad (50)$$

$$b = 2(\mathbf{o} - \mathbf{c}) \cdot \mathbf{d} \quad (51)$$

$$c = \|\mathbf{o} - \mathbf{c}\|^2 - r^2 \quad (52)$$

Дискриминант:

$$\Delta = b^2 - 4ac = b^2 - 4c$$

Если $\Delta < 0$, пересечений нет. Иначе:

$$t = \frac{-b \pm \sqrt{\Delta}}{2a} = \frac{-b \pm \sqrt{\Delta}}{2}$$

Выбирается минимальное положительное t в диапазоне $[t_{\min}, t_{\max}]$.

Нормаль в точке пересечения:

$$\mathbf{n} = \frac{\mathbf{r}(t) - \mathbf{c}}{r}$$

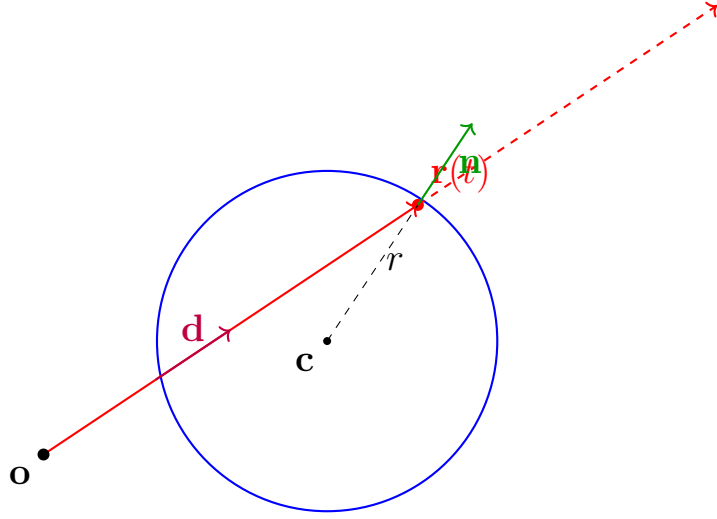


Рис. 2: Пересечение луча со сферой

5.2 Пересечение с плоскостью

Плоскость задается точкой $\mathbf{p}$ и нормалью $\mathbf{n}$ (нормализована). Уравнение плоскости:

$$(\mathbf{x} - \mathbf{p}) \cdot \mathbf{n} = 0$$

Подставляя параметрическое уравнение луча:

$$(\mathbf{o} + t\mathbf{d} - \mathbf{p}) \cdot \mathbf{n} = 0 \quad (53)$$

$$(\mathbf{o} - \mathbf{p}) \cdot \mathbf{n} + t(\mathbf{d} \cdot \mathbf{n}) = 0 \quad (54)$$

Отсюда:

$$t = \frac{(\mathbf{p} - \mathbf{o}) \cdot \mathbf{n}}{\mathbf{d} \cdot \mathbf{n}}$$

Если $|\mathbf{d} \cdot \mathbf{n}| < \epsilon$ (луч параллелен плоскости), пересечения нет. Иначе проверяется, что $t \in [t_{\min}, t_{\max}]$.

Точка пересечения:

$$\mathbf{hit} = \mathbf{o} + t\mathbf{d}$$

Нормаль: $\mathbf{n}$ (уже известна).

5.3 Пересечение с кубом (AABB)

Ограничивающий параллелепипед (Axis-Aligned Bounding Box, AABB) задается двумя точками $\mathbf{b}_{\min}$ и $\mathbf{b}_{\max}$.

Для каждой оси $i \in \{x, y, z\}$ вычисляются параметры пересечения:

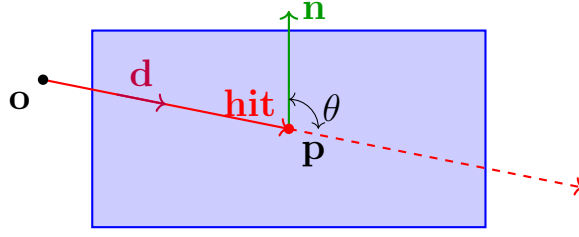


Рис. 3: Пересечение луча с плоскостью

$$t_{\text{near},i} = \frac{b_{\text{min},i} - o_i}{d_i} \quad (55)$$

$$t_{\text{far},i} = \frac{b_{\text{max},i} - o_i}{d_i} \quad (56)$$

Если $d_i < 0$, луч идет в обратном направлении, поэтому значения меняются местами:

$$\text{if } d_i < 0 : \quad t_{\text{near},i} \leftrightarrow t_{\text{far},i} \quad (57)$$

Итоговые параметры пересечения:

$$t_{\text{near}} = \max(t_{\text{near},x}, t_{\text{near},y}, t_{\text{near},z}) \quad (58)$$

$$t_{\text{far}} = \min(t_{\text{far},x}, t_{\text{far},y}, t_{\text{far},z}) \quad (59)$$

Пересечение существует, если:

$$t_{\text{near}} < t_{\text{far}} \quad \text{и} \quad t_{\text{far}} > 0$$

Параметр пересечения: $t = t_{\text{near}}$ (если $t_{\text{near}} > 0$) или $t = t_{\text{far}}$ (если луч начинается внутри куба).

Нормаль определяется по ближайшей грани:

$$\mathbf{n}_i = \begin{cases} (1, 0, 0) & \text{если ближайшая грань по } x \\ (0, 1, 0) & \text{если ближайшая грань по } y \\ (0, 0, 1) & \text{если ближайшая грань по } z \end{cases}$$

с соответствующим знаком в зависимости от направления.

6 Материалы и BRDF

6.1 Уравнение рендеринга

Уравнение рендеринга описывает количество света, исходящего из точки $\mathbf{x}$ в направлении ω_o :

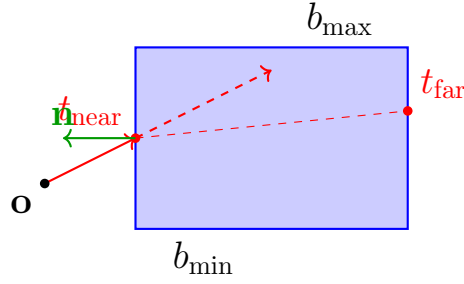


Рис. 4: Пересечение луча с AABB (кубом)

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} f_r(\mathbf{x}, \omega_i, \omega_o) L_i(\mathbf{x}, \omega_i) (\mathbf{n} \cdot \omega_i) d\omega_i$$

где:

- L_o — исходящая яркость (outgoing radiance)
- L_e — эмиссия (emission)
- f_r — BRDF (Bidirectional Reflectance Distribution Function)
- L_i — входящая яркость (incoming radiance)
- $\mathbf{n}$ — нормаль поверхности
- Ω — полусфера направлений над поверхностью

6.2 Lambertian (диффузный) материал

Lambertian BRDF моделирует идеально рассеивающую поверхность:

$$f_r(\omega_i, \omega_o) = \frac{\rho}{\pi}$$

где ρ — альбедо (albedo), цвет материала.

Это означает, что поверхность отражает свет равномерно во все направления, независимо от направления падения и наблюдения.

6.3 Металлический материал — GGX распределение

Для металлических материалов используется микрофасетная модель с распределением нормалей GGX (Trowbridge-Reitz):

$$D(\mathbf{h}) = \frac{\alpha^2}{\pi((\mathbf{n} \cdot \mathbf{h})^2(\alpha^2 - 1) + 1)^2}$$

где:

- $\mathbf{h} = \frac{\omega_i + \omega_o}{\|\omega_i + \omega_o\|}$ — половинный вектор (half vector)
- $\alpha = \text{roughness}^2$ — параметр шероховатости
- $\mathbf{n}$ — нормаль поверхности

Для идеального зеркала (mirror) используется простое отражение:

$$\omega_o = \text{reflect}(\omega_i, \mathbf{n}) = \omega_i - 2(\omega_i \cdot \mathbf{n})\mathbf{n}$$

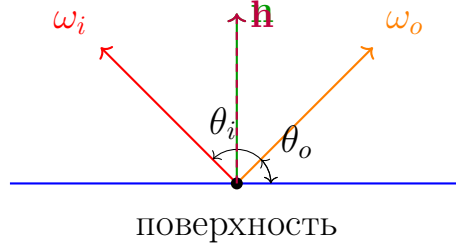


Рис. 5: Отражение луча от поверхности (зеркало/металл)

6.4 Диэлектрический материал — закон Снеллиуса и Френель

Для прозрачных материалов (стекло, вода) используется закон Снеллиуса:

$$\eta_1 \sin \theta_1 = \eta_2 \sin \theta_2$$

где η_1, η_2 — показатели преломления сред, θ_1, θ_2 — углы падения и преломления.

Вектор преломления вычисляется как:

$$\cos \theta_1 = -\omega_i \cdot \mathbf{n} \quad (60)$$

$$\sin \theta_1 = \sqrt{1 - \cos^2 \theta_1} \quad (61)$$

$$\sin \theta_2 = \frac{\eta_1}{\eta_2} \sin \theta_1 \quad (62)$$

Если $\sin \theta_2 > 1$, происходит полное внутреннее отражение (TIR), и луч отражается.

Иначе:

$$\cos \theta_2 = \sqrt{1 - \sin^2 \theta_2} \quad (63)$$

$$\mathbf{r}_\perp = \eta(\omega_i + \cos \theta_1 \mathbf{n}) \quad (64)$$

$$\mathbf{r}_\parallel = -\sqrt{1 - \|\mathbf{r}_\perp\|^2} \mathbf{n} \quad (65)$$

$$\omega_o = \mathbf{r}_\perp + \mathbf{r}_\parallel \quad (66)$$

где $\eta = \eta_1/\eta_2$ (при входе) или $\eta = \eta_2/\eta_1$ (при выходе).

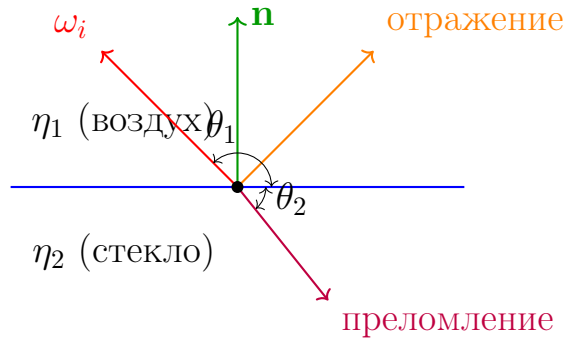
Коэффициент отражения Френеля (аппроксимация Шлика):

$$R(\theta) = R_0 + (1 - R_0)(1 - \cos \theta)^5$$

где:

$$R_0 = \left(\frac{\eta_2 - \eta_1}{\eta_2 + \eta_1} \right)^2$$

Вероятность отражения используется для выбора между отражением и преломлением методом Монте-Карло.



$$\eta_1 \sin \theta_1 = \eta_2 \sin \theta_2$$

Рис. 6: Преломление и отражение (закон Снеллиуса)

6.5 Поглощение света (Beer-Lambert закон)

При прохождении через диэлектрик свет поглощается по закону Бугера-Ламберта:

$$I(d) = I_0 \exp(-\sigma d)$$

где:

- I_0 — начальная интенсивность
- $\sigma = \boldsymbol{\alpha} \times \text{absorptionScale}$ — коэффициент поглощения (вектор для RGB)
- d — пройденное расстояние

В коде это реализуется как:

$$\text{attenuation} = \exp(-\alpha \times \text{scale} \times d)$$

7 Importance Sampling

7.1 Cosine-weighted sampling для диффузных материалов

Для эффективного сэмплирования диффузных материалов используется косинусное распределение:

$$p(\omega) = \frac{\cos \theta}{\pi}$$

где θ — угол между направлением и нормалью.

Сэмплирование в локальной системе координат:

$$\phi = 2\pi\xi_1 \quad (\text{равномерно по азимуту}) \quad (67)$$

$$\cos \theta = \sqrt{\xi_2} \quad (\text{косинусное распределение}) \quad (68)$$

$$\sin \theta = \sqrt{1 - \xi_2} \quad (69)$$

где $\xi_1, \xi_2 \sim \mathcal{U}(0, 1)$.

Направление в локальной системе:

$$\mathbf{d}_{\text{local}} = (\sin \theta \cos \phi, \sin \theta \sin \phi, \cos \theta)$$

Преобразование в мировую систему через ортонормированный базис $(\mathbf{u}, \mathbf{v}, \mathbf{n})$:

$$\mathbf{d}_{\text{world}} = \mathbf{d}_{\text{local},x}\mathbf{u} + \mathbf{d}_{\text{local},y}\mathbf{v} + \mathbf{d}_{\text{local},z}\mathbf{n}$$

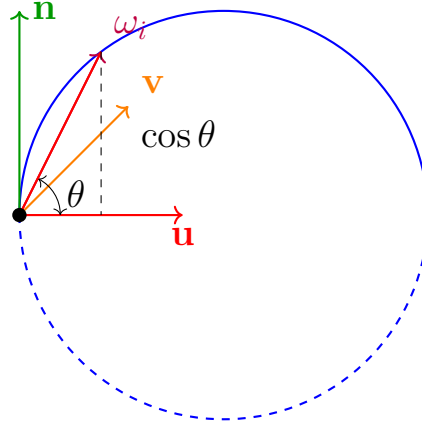
PDF по направлению:

$$p(\omega) = \frac{\cos \theta}{\pi}$$

7.2 GGX importance sampling для металлов

Для GGX распределения используется importance sampling нормалей микроповерхности:

$$\cos \theta_h = \sqrt{\frac{1 - \xi_2}{1 + (\alpha^2 - 1)\xi_2}} \quad (70)$$



Плотность: $p(\omega) = \frac{\cos \theta}{\pi}$

Рис. 7: Cosine-weighted sampling для диффузных материалов

$$\sin \theta_h = \sqrt{1 - \cos^2 \theta_h} \quad (71)$$

$$\phi_h = 2\pi\xi_1 \quad (72)$$

Половинный вектор в локальной системе:

$$\mathbf{h}_{\text{local}} = (\sin \theta_h \cos \phi_h, \sin \theta_h \sin \phi_h, \cos \theta_h)$$

Преобразование в мировую систему и вычисление направления отражения:

$$\omega_o = \text{reflect}(-\omega_i, \mathbf{h})$$

7.3 Next Event Estimation (NEE)

Для уменьшения дисперсии используется прямое сэмплирование источников света (Next Event Estimation).

Вклад от источника света:

$$L_{\text{direct}} = \frac{f_r(\omega_i, \omega_o) L_e \cos \theta_i \cos \theta_o}{r^2 \cdot p_A}$$

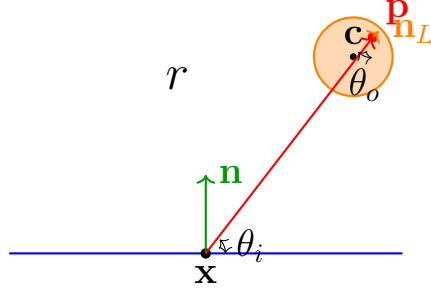
где:

- L_e — эмиссия источника
- $\cos \theta_i$ — косинус угла между нормалью поверхности и направлением к источнику
- $\cos \theta_o$ — косинус угла между нормалью источника и направлением от источника

- r — расстояние до источника
- $p_A = 1/A$ — PDF по площади источника (для равномерного сэмпирования)

Переход от PDF по площади к PDF по направлению:

$$p(\omega) = \frac{p_A \cos \theta_o}{r^2}$$



$$\text{Геометрия: } \frac{\cos \theta_i \cos \theta_o}{r^2}$$

Рис. 8: Next Event Estimation (NEE) — прямое сэмпирование источника света

7.4 Сэмпирование точки на сфере-источнике

Для сферического источника света точка сэмпировается равномерно по поверхности:

$$z = 1 - 2\xi_1 \quad (\text{равномерно по } z) \quad (73)$$

$$r = \sqrt{1 - z^2} \quad (74)$$

$$\phi = 2\pi\xi_2 \quad (75)$$

$$\mathbf{p}_{\text{local}} = (r \cos \phi, r \sin \phi, z) \quad (76)$$

$$\mathbf{p}_{\text{world}} = \mathbf{c} + R \cdot \frac{\mathbf{p}_{\text{local}}}{\|\mathbf{p}_{\text{local}}\|} \quad (77)$$

где R — радиус сферы, $\mathbf{c}$ — центр сферы.
PDF по площади:

$$p_A = \frac{1}{4\pi R^2}$$

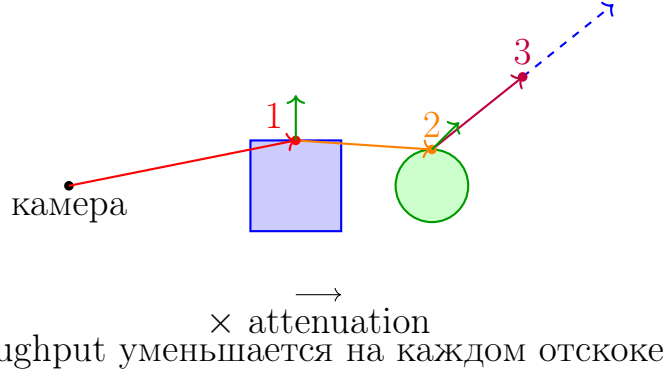


Рис. 9: Path tracing — трассировка случайного пути света

8 Path Tracing алгоритм

8.1 Рекурсивное уравнение рендеринга

Path tracing решает уравнение рендеринга методом Монте-Карло, трассируя случайные пути света:

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} f_r(\mathbf{x}, \omega_i, \omega_o) L_i(\mathbf{x}, \omega_i) (\mathbf{n} \cdot \omega_i) d\omega_i$$

Метод Монте-Карло аппроксимирует интеграл:

$$L_o \approx L_e + \frac{1}{N} \sum_{i=1}^N \frac{f_r(\omega_i, \omega_o) L_i(\omega_i) (\mathbf{n} \cdot \omega_i)}{p(\omega_i)}$$

где $p(\omega_i)$ — PDF сэмплирования направления.

8.2 Итеративный алгоритм path tracing

Вместо рекурсии используется итеративный подход:

$$\text{throughput} = (1, 1, 1) \quad (\text{начальное значение}) \quad (78)$$

$$\text{radiance} = (0, 0, 0) \quad (79)$$

$$\text{depth} = \text{maxDepth} \quad (80)$$

Для каждого отскока:

$$\text{radiance} \leftarrow \text{radiance} + \text{throughput} \times L_e \quad (\text{если материал эмиссивный}) \quad (81)$$

$$\text{radiance} \leftarrow \text{radiance} + \text{throughput} \times L_{\text{direct}} \quad (\text{NEE}) \quad (82)$$

$$\omega_i \sim p(\omega) \quad (\text{сэмплирование нового направления}) \quad (83)$$

$$\text{throughput} \leftarrow \text{throughput} \times \frac{f_r(\omega_i, \omega_o)(\mathbf{n} \cdot \omega_i)}{p(\omega_i)} \quad (84)$$

$$\text{depth} \leftarrow \text{depth} - 1 \quad (85)$$

8.3 Russian Roulette

Для раннего прекращения трассировки используется Russian Roulette:

$$\text{continue} = \begin{cases} \text{true} & \text{с вероятностью } p = \min(\max(\text{throughput}), 0.95) \\ \text{false} & \text{с вероятностью } 1 - p \end{cases}$$

Если трассировка продолжается, throughput корректируется:

$$\text{throughput} \leftarrow \frac{\text{throughput}}{p}$$

Это обеспечивает несмещенность оценки (unbiased estimation).

8.4 Условия прекращения

Трассировка прекращается, если:

- $\text{depth} \leq 0$ — достигнута максимальная глубина
- Луч не пересекает объекты (уходит в небо)
- Russian Roulette решает прекратить трассировку
- $\max(\text{throughput}) < \epsilon$ — throughput стал слишком мал

9 Объемное рассеяние (Volumetric Scattering)

9.1 Уравнение переноса излучения

Уравнение переноса излучения описывает изменение яркости вдоль луча в объемной среде:

$$\frac{dL(\mathbf{x}, \omega)}{ds} = -\sigma_t(\mathbf{x})L(\mathbf{x}, \omega) + \sigma_s(\mathbf{x}) \int_{4\pi} p(\omega_i, \omega) L_i(\mathbf{x}, \omega_i) d\omega_i$$

где:

- $\sigma_t = \sigma_s + \sigma_a$ — коэффициент экстинкции
- σ_s — коэффициент рассеяния
- σ_a — коэффициент поглощения
- $p(\omega_i, \omega)$ — фазовая функция

9.2 Фазовая функция Хеньи-Грина

Фазовая функция Хеньи-Грина (Henyeu-Greenstein) моделирует рассеяние в объемной среде:

$$p(\cos \theta) = \frac{1 - g^2}{4\pi(1 + g^2 - 2g \cos \theta)^{3/2}}$$

где:

- $g \in [-1, 1]$ — параметр анизотропии
- $g = 0$ — изотропное рассеяние
- $g > 0$ — рассеяние вперед (forward scattering)
- $g < 0$ — рассеяние назад (backward scattering)

9.3 Single Scattering интеграл

Для single scattering (однократное рассеяние) интеграл упрощается. Вклад от источника света в точке $\mathbf{x}$:

$$L_s(\mathbf{x}, \omega) = \int_0^d T(\mathbf{x}, \mathbf{x} + s\omega) \sigma_s(\mathbf{x} + s\omega) \int_{4\pi} p(\omega_i, \omega) L_e(\mathbf{x} + s\omega, \omega_i) d\omega_i ds$$

где T — функция пропускания (transmittance):

$$T(\mathbf{a}, \mathbf{b}) = \exp \left(- \int_0^{\|\mathbf{b}-\mathbf{a}\|} \sigma_t(\mathbf{a} + t \frac{\mathbf{b} - \mathbf{a}}{\|\mathbf{b} - \mathbf{a}\|}) dt \right)$$

Для однородной среды:

$$T(d) = \exp(-\sigma_t d)$$

9.4 Численное интегрирование

Интеграл аппроксимируется методом прямоугольников:

$$L_s \approx \sum_{i=0}^{N-1} T(s_i) \sigma_s(\mathbf{x}_i) p(\omega_i, \omega) L_e(\mathbf{x}_i, \omega_i) \Delta s$$

где:

- $s_i = (i + 0.5)\Delta s$ — позиция сэмпла
- $\Delta s = d/N$ — шаг интегрирования

- $\mathbf{x}_i = \mathbf{x} + s_i \omega$ — позиция сэмпла
- N — количество шагов (обычно 24)

9.5 Beer-Lambert закон для поглощения

При прохождении через среду свет ослабляется по закону Бугера-Ламберта:

$$L(d) = L_0 \exp(-\sigma_a d)$$

где d — пройденное расстояние.

Для неоднородной среды:

$$L(d) = L_0 \exp \left(- \int_0^d \sigma_a(\mathbf{x} + t\omega) dt \right)$$

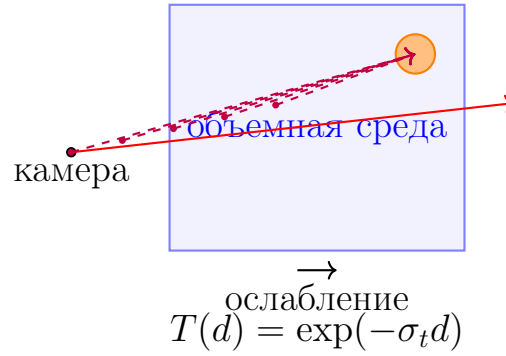


Рис. 10: Объемное рассеяние — single scattering в тумане

10 Накопление и усреднение сэмплов

10.1 Накопление сэмплов

Для каждого пикселя накапливается сумма всех сэмплов:

$$\text{accum}[i] = \sum_{s=1}^N L_s(i)$$

где:

- $i = y \times W + x$ — линейный индекс пикселя
- $L_s(i)$ — яркость s -го сэмпла для пикселя i
- N — общее количество сэмплов

В коде это реализуется как:

$$\text{accumData}[i \times 3 + 0] \leftarrow \text{accumData}[i \times 3 + 0] + L_s(i).r \quad (86)$$

$$\text{accumData}[i \times 3 + 1] \leftarrow \text{accumData}[i \times 3 + 1] + L_s(i).g \quad (87)$$

$$\text{accumData}[i \times 3 + 2] \leftarrow \text{accumData}[i \times 3 + 2] + L_s(i).b \quad (88)$$

10.2 Усреднение

Итоговый цвет пикселя вычисляется как среднее арифметическое:

$$\bar{L}(i) = \frac{1}{N} \sum_{s=1}^N L_s(i) = \frac{\text{accum}[i]}{N}$$

где $N = \text{sampleCount}$ — текущее количество накопленных сэмплов.
Компоненты цвета:

$$\bar{L}_r(i) = \frac{\text{accumData}[i \times 3 + 0]}{N} \quad (89)$$

$$\bar{L}_g(i) = \frac{\text{accumData}[i \times 3 + 1]}{N} \quad (90)$$

$$\bar{L}_b(i) = \frac{\text{accumData}[i \times 3 + 2]}{N} \quad (91)$$

10.3 Запись в текстуру

Итоговый цвет записывается в текстуру формата RGBA16F (16-битное число с плавающей точкой на канал):

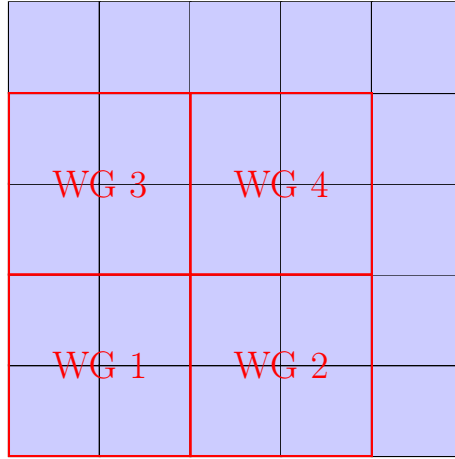
$$\text{imageStore}(\text{destTex}, \text{pix}, (\bar{L}_r, \bar{L}_g, \bar{L}_b, 1.0))$$

Формат RGBA16F позволяет хранить значения вне диапазона $[0, 1]$, что важно для HDR (High Dynamic Range) рендеринга.

11 Оптимизации

11.1 Кэширование вычислений

Для ускорения вычислений используются оптимизации:



Work groups (16×16 потоков каждый)
Каждый пиксель обрабатывается независимо

Рис. 11: Распараллеливание: work groups и обработка пикселей

11.1.1 Быстрая нормализация

Вместо стандартной нормализации используется оптимизированная версия:

$$\mathbf{v}_{\text{norm}} = \mathbf{v} \times \text{inversesqrt}(\|\mathbf{v}\|^2)$$

где `inversesqrt` — быстрая инструкция GPU для вычисления $1/\sqrt{x}$.

11.1.2 Кэширование квадратов

Часто используемые квадраты вычисляются один раз:

$$r^2 = r \times r, \quad \alpha^2 = \alpha \times \alpha$$

11.1.3 Кэширование обратных значений

Обратные значения вычисляются один раз:

$$\text{invA} = \frac{1}{a}, \quad \text{invRadius} = \frac{1}{r}$$

11.2 Ранний выход из циклов

Для оптимизации используются ранние выходы:

- Если луч не пересекает объект, цикл прерывается
- Если throughput стал слишком мал, трассировка прекращается

- Если материал эмиссивный, трассировка прекращается после добавления эмиссии

11.3 Оптимизация доступа к памяти

- Последовательный доступ к массивам объектов и материалов
- Кэширование часто используемых значений в регистрах
- Минимизация количества обращений к глобальной памяти

11.4 Firefly Reduction

Для уменьшения артефактов (fireflies) от очень ярких сэмплов используется ограничение яркости:

$$L_{\text{clamped}} = \begin{cases} L & \text{если } \text{luminance}(L) \leq L_{\text{max}} \\ L \times \frac{L_{\text{max}}}{\text{luminance}(L)} & \text{иначе} \end{cases}$$

где:

$$\text{luminance}(L) = 0.2126 \times L_r + 0.7152 \times L_g + 0.0722 \times L_b$$

и $L_{\text{max}} = 500.0$ — максимальная допустимая яркость.

12 Заключение

В данном документе представлено полное математическое обоснование работы патрейсера на видеокарте. Описаны:

- Форматы данных и их передача на GPU через UBO и SSBO
- Механизм распараллеливания через compute shaders
- Математические формулы для генерации лучей и пересечений
- BRDF для различных типов материалов
- Алгоритмы importance sampling
- Path tracing с Russian Roulette
- Объемное рассеяние света
- Накопление и усреднение сэмплов

- Оптимизации для повышения производительности

Все формулы выведены из физических принципов и соответствуют реализации в коде.